Transients Without Tears: Automatic In-Place Conversion of Persistent Data Structure Updates in a Dynamic Language

Anonymous Author(s)

Abstract

Persistent-first languages pay a large allocation tax on accumulation loops. Languages like Koka eliminate it with ownership types (Perceus/FBIP); Clojure asks the programmer to opt in manually with transients. We show the optimization can be made *automatic and sound* in a latently-typed, open-world language with live redefinition—with no type system—via (1) a tail-position-sensitive *usage classification* that proves the *linear usage* of loop and fold accumulators, (2) a rewriter aligned with the classifier *by construction*, targeting edit-tagged in-place transients, and (3) *world-guarded regions* that keep the optimization sound when definitions change at runtime. The result: 4.1–7.6x on accumulation-bound code, byte-identical outputs on a production-scale event simulator, zero measured cost when disabled or inapplicable, and an honest cost model of when transient conversion pays.

CCS Concepts: • **Software and its engineering → Compilers**; **Dynamic compilers**; • **Theory of computation → Program analysis**.

Keywords: escape analysis, persistent data structures, transients, in-place update, dynamic languages, lisp, speculative optimization, deoptimization

ACM Reference Format:
Anonymous Author(s). 2027. Transients Without Tears: Automatic In-Place Conversion of Persistent Data Structure Updates in a Dynamic Language. In *Proceedings of the 48th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '27).* ACM, New York, NY, USA, 4 pages. https://doi.org/XXXXXXX.XXXXXXX

PLDI '27, TBD

ACM ISBN 978-1-4503-XXXX-X/27/06
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Persistent data structures impose a significant performance tax. In FOL, our Lisp dialect, accumulation-heavy benchmarks can run up to 20× slower and allocate **900×** more memory than their mutable counterparts, dominated by garbage collection pressure. The canonical pattern is an accumulator in a loop, such as `(loop [acc {}] ... (recur (assoc acc k v)))` or a `reduce`, where each update allocates a new version of the collection.

Existing solutions do not transfer to our setting. Perceus/FBIP [6] requires ownership types, which FOL lacks. Clojure's transients are a manual, unsafe escape hatch. Crucially, FOL is a dynamic, open-world language where method and class definitions can change while optimized code is live, precluding any static analysis that assumes a closed world.

This paper shows how to make transient conversion automatic and sound in this hostile environment. Our contributions are:

1. **Linear usage analysis.** A flow- and tail-position-sensitive analysis that proves an accumulator is used linearly—that is, uniquely at the point of update—and can therefore be safely converted to an in-place transient, without requiring a formal type system.
2. **Alignment-by-construction.** The analysis and transformation are a single pass, ensuring that only provably safe patterns are rewritten.
3. **Name-resolution-faithful summaries.** A table of standard library operator effects that mirrors the compiler's own name resolution, ensuring the analysis is as sound as the language's semantics.
4. **Soundness under live redefinition.** A world-guard mechanism emits dual-path code that safely falls back to the persistent path if a function or class assumed by the optimization is redefined at runtime.
5. **An honest empirical cost model.** We evaluate the optimization on microbenchmarks and a real-world event simulator, quantifying both the wins (4-7x) and the limitations, including a motivating benchmark that our intra-procedural analysis cannot convert.

2 Background and Motivating Example

FOL is a Lisp-1 dialect with persistent vectors (32-way tries), HAMT dictionaries, and a transpiler to Common Lisp. Standard library functions are resolved by name at emit time.

Consider the `lh-pop-batch` function from our event simulator benchmark. It drains a queue, building up a result vector with `conj`.

```
;; --- Source ---
(defn lh-pop-batch [q n]
  (loop [acc [] q q i 0]
    (if (or (>= i n) (empty? q))
      [acc q]  ; acc exits inside a literal
      (recur (conj acc (peek q)) (pop q) (
          inc i)))))

;; --- After transient conversion ---
(let [validity-cell (register-region '("conj
    " "peek" "pop"))]
  (if (car validity-cell)
    ;; Fast path
    (loop [acc (transient []) q q i 0]
      (if (or (>= i n) (empty? q))
        [(persistent! acc) q]
        (recur (conj! acc (peek q)) (pop q)
            (inc i))))
    ;; Fallback path (original code)
    ...))
```

Our system automatically converts this loop to use a transient accumulator. The `conj` becomes a mutating `conj!`, and the accumulator is sealed with `persistent!` at the exit. The entire fast path is wrapped in a world guard that checks if any of the assumed functions have been redefined.

3 The Analysis

Our analysis proves **linear usage**, a property analogous to that enforced by linear type systems [7] but established here by a syntactic usage analysis. A value is used linearly if it is fresh, unaliased, and every use is a last-use that flows into a valid update or exit.

3.1 Tier-1 Summaries

We use a hand-verified table of 75 standard library function summaries. Each summary describes the function's effect on its arguments using a simple lattice: `:none` < `:invoked` < `:shared-with-result` < `:retained`. A key assumption is the **root-level convention**: persistent operations like `assoc` produce a fresh root node, which is safe for all clients because our transient protocol is copy-on-write.

3.2 Name-Resolution Faithfulness

The analysis is only as sound as its knowledge of what function is being called. Our summary table mirrors the compiler's own name resolution logic, including per-file lexical definitions. This leads to a key lemma: any program the analysis misjudges is a program the compiler itself would have resolved the same way.

3.3 The Classifier

The core of the analysis is a recursive walk over the AST that classifies every use of a candidate accumulator. It propagates tail-position context through `if`, `do`, `bind`, etc. It recognizes sanctioned uses like being the argument to an update function (`conj`, `assoc`) or appearing at a valid exit point. Any other use, such as being stored in another data structure or captured in a closure, disqualifies the accumulator.

3.4 Chains

The analysis recognizes chains of linear updates, including direct calls, `->` threading macros, and if-branched chains. It also handles **reduce init-threading**: a `reduce` whose lambda is proven to be linear becomes a valid link in an update chain itself.

3.5 Formal Core

We formalize the analysis for a core calculus. The key judgment, ‘; e U‘, states that expression ‘e‘ has usage ‘U‘ in context ‘‘ given operator summaries ‘‘. We show that "messy" implementation features are sound elaborations of this core: ‘->‘ threads and ‘if‘-branched chains desugar into nested applications that the core rules handle, and name-resolution faithfulness is a property of how ‘‘ is constructed, not the rules themselves. The main theorem (proof sketched in an appendix) states that if an accumulator qualifies, the transient-converted program is observationally equivalent to the original, proven via bisimulation against a store semantics. The termination and correctness of the inter-procedural fixpoint analysis are discussed formally in Appendix A.

4 Transformation and Representation

The transformation is aligned by construction with the analysis.

4.1 Rewriter

If an accumulator qualifies, the rewriter wraps its initializer with `transient`, replaces all update calls with their ‘¡ counterparts (e.g., `conj` -> `conj!`), and inserts `persistent!` at all valid exit points.

4.2 Edit-Tagged Transients

Our implementation uses edit-tagged transients, a token-based copy-on-write (COW) HAMT for dictionaries and a vector with an owned tail. This is crucial as it allows reads on transients, unlike simpler wrapper-based approaches. A plain persistent lookup would silently misread edited data; our transient-aware read primitives correctly handle lookups on the mutable view.

5 Soundness under Live Redefinition

The primary threat to soundness is live redefinition. If our optimization bakes in the meaning of `assoc`, a later `(defn assoc ...)` could invalidate the compiled code.

Our solution is a dual-path emission strategy. At load time, each optimized region registers its dependencies (e.g., ‘"assoc", "conj"‘) and receives a validity cell. At execution time, defining a function like `assoc` calls `note-redefinition`, which looks up all dependent regions and flips their validity cells. The compiled code checks this cell on entry.

```
(IF (CAR (LOAD-TIME-VALUE (REGISTER-REGION
    '("assoc"))))
    ;; Fast path: use assoc!
    ...
    ;; Fallback path: use original
        persistent assoc
    ...)
```

This mechanism is a static, region-scoped analogue to a JIT's world-age or deoptimization system, but requires no runtime recompilation.

6 Implementation

The system is implemented in 2,300 lines of Common Lisp across the analysis, rewriter, and world-guard machinery. It includes an "audit mode" that runs the analysis without emitting code. With only intra-procedural (Tier-1) analysis, the audit found 9 convertible candidates out of 17 loops in our LSim benchmark. The DVI benchmark, a key motivator whose accumulator is passed through a user function, had 0 candidates. With the addition of our inter-procedural (Tier-2) summary inference engine, the DVI benchmark's helper function is correctly summarized, and its main loop now converts successfully.

7 Evaluation

We evaluate correctness, performance on microbenchmarks, and end-to-end impact. All benchmarks were run on a Ryzen 9 5900X with SBCL 2.6.0.

RQ1 (Correctness): Our full test suite of 3,248 checks passes with the optimization enabled. The LSim event simulator produces byte-identical output.

RQ2 (Speedup): On accumulation-bound microbenchmarks, we see speedups of 4.1x to 7.6x and memory allocation reductions of 8-11x.

Table 1. Microbenchmark results (best-of-3).

Workload	Baseline	Optimized	Speedup	Mem. Reduction
dict loop (200k)	224.6ms	55.2ms	**4.07×**	8.7x
vector loop (1M)	309.9ms	41.0ms	**7.56×**	11.0x
reduce (500k)	324.1ms	48.8ms	**6.64×**	9.1x
DVI (200k)	241.2ms	63.1ms	**3.82×**	8.5x

RQ3 (End-to-end): On the LSim simulator, the wall-clock speedup is modest (1.04x), though allocation is reduced by 1.10x. This is explained by our cost model: the hot loops in LSim perform only small-N accumulations, where the constant-factor overhead of the transient boundaries limits the win. The dominant cost is heap churn from other sources, which our technique does not target.

RQ4 (Cost): When disabled or inapplicable, the performance is identical to the baseline within measurement noise. The world guards are effectively free.

8 Discussion and Limitations

Our inter-procedural analysis significantly broadens the optimization's reach, as demonstrated by the successful conversion of the DVI benchmark. However, limitations remain.

Inter-procedural Analysis Scope. Our summary inference engine operates on a file-at-a-time basis. While it can analyze through any chain of calls within a single compilation unit, it cannot see across module boundaries. A function imported from another module is treated as a black box, and passing an accumulator to it will disqualify the conversion. A whole-program analysis or a system of explicit, checked interface contracts would be required to lift this limitation.

Profitability Heuristics. As our end-to-end evaluation on LSim shows, the performance gain on small-N accumulations is bounded by the constant-factor overhead of creating and sealing transient collections. For loops with very few iterations, this can lead to a performance regression. A profitability heuristic could prevent this by augmenting the world guard with a runtime check, such as ‘(> (count accumulator) <threshold>)‘, where the threshold is an empirically determined constant. This would ensure the optimization is only applied when likely to be beneficial, making it safer to enable by default.

Finally, our transient representation for sets does not yet support reads. This is an implementation limitation: due to the unordered nature of the underlying hash table, providing a consistent copy-on-write view for reads is more complex than for indexed collections. This restricts the optimization's applicability for set-based accumulators in read-heavy loops.

9 Related Work

Our work is an alternative to type-system-based approaches like Perceus/FBIP [5, 6] for languages that lack ownership or linear types, and automates the manual process of using Clojure's transients [4]. Unlike classic escape analysis [2, 3], which is a *may-analysis* for stack allocation, our linear usage analysis is a *must-analysis* for proving the safety of in-place mutation.

Our world-guard mechanism is a static, ahead-of-time (AOT) analogue to the assumption-and-deoptimization systems used in Just-in-Time (JIT) compilers. The choice of a static approach is dictated by FOL's nature as a transpiled

AOT language, which lacks the runtime infrastructure for JIT-style dynamic recompilation. A JIT might speculatively optimize a loop assuming a function is not redefined, and then deoptimize back to an interpreter if that assumption is violated. Our approach, in contrast, emits both the fast and fallback paths ahead of time, selecting between them with a simple guard.

This AOT strategy has distinct advantages: it avoids the complexity and runtime overhead of a JIT, including on-stack replacement machinery and interpreter frame metadata. The performance is predictable, with no stalls from runtime recompilation. The primary trade-off is granularity: our invalidation occurs at region entry, whereas a JIT can deoptimize a running activation mid-loop. Our system is most similar to Julia's world-age mechanism [1], but we apply it to uniqueness and escape facts rather than method dispatch tables.

10 Conclusion

We have presented a system for automatically converting allocation-heavy loops over persistent data structures to use in-place transient updates. Our approach is sound in a dynamic, open-world language, combining a syntactic linear usage analysis with a world-guard mechanism to handle live redefinition. The result is a significant performance improvement for accumulation-bound code, achieved without programmer annotation or a formal type system.

References

[1] Kateryna Belyakova, Jeff Bezanson, Alan Edelman, Steven G Johnson, and Viral B Shah. 2020. Julia's world age: Soundness and performance in a dynamic language. In *Proceedings of the ACM on Programming Languages*, Vol. 4. ACM New York, NY, USA, 1–29.

[2] Bruno Blanchet. 2003. Escape analysis for object-oriented languages. application to Java. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 25, 6 (2003), 713–775.

[3] Jong-Deok Choi, Manish Gupta, Mauricio Serrano, Vugranam C Sreedhar, and Samuel Midkiff. 1999. Escape analysis for Java. In *Acm sigplan notices*, Vol. 34. ACM New York, NY, USA, 1–19.

[4] Rich Hickey. 2009. The Clojure programming language. *Presentation at the 2009 JVM Language Summit* (2009).

[5] Christian Lorenzen and Daan Leijen. 2023. Functional but in-place: The case for borrowing in functional languages. In *Proceedings of the 8th ACM SIGPLAN International Symposium on Principles and Practice of Declarative Programming*.

[6] Alex Reinking, Daan Koka, and Daan Leijen. 2021. Perceus: Garbage free reference counting with reuse. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. 664–679.

[7] Philip Wadler. 1990. Linear types can change the world!. In *IFIP TC 2 Working Conference on Programming Concepts and Methods*. North-Holland.

A Fixpoint Analysis Formalism

Our inter-procedural analysis for inferring function summaries (Tier-2) operates by iterating to a fixpoint. This appendix provides a formal sketch of this process, establishing its termination and correctness.

A.1 The Summary Lattice

The analysis computes a usage summary for each function. A summary for a function with k parameters is a tuple $(u_1, u_2, \ldots, u_k)$, where each u_i is an element from the finite usage lattice $\mathcal{U}$:

$$\mathcal{U} = \{\texttt{:none} \sqsubset \texttt{:invoked} \sqsubset \texttt{:shared-with-result} \sqsubset \texttt{:retained}\}$$

Here, $\sqsubset$ denotes "is more precise than". The top element, `:retained`, is the most conservative assumption (the parameter may be retained indefinitely), while `:none` is the most precise (the parameter is not used). The lattice of summaries for a single function is the product lattice $\mathcal{U}^k$, which is also finite.

A.2 Transfer Function and Fixpoint Iteration

Let S be the set of all function summaries in a compilation unit (a file). We define a transfer function $F : S \to S$ that computes a new set of summaries based on the current ones. For each function f in the unit, F re-analyzes its body using the current summaries for all functions it calls.

The analysis proceeds by iterating $S_{i+1} = F(S_i)$, starting with the most conservative assumption, S_0, where all parameters of all user-defined functions are marked as `:retained`.

Termination. The analysis is guaranteed to terminate because:

1. The overall summary space for the compilation unit is finite, as it is the product of the finite summary lattices for each function.
2. The transfer function F is monotonic. As the analysis gains more precise information about a callee (i.e., its summary moves down the lattice), the inferred summary for the caller can only become more precise or stay the same. It can never become less precise (move up the lattice).

Since the iteration proceeds over a finite lattice with a monotonic function, it is guaranteed to reach a fixpoint in a finite number of steps. In practice, for typical call graphs, this fixpoint is reached very quickly (2-3 iterations).